Do Verified Trajectories Improve Software-Agent Reliability?

A Pilot Study of Trajectory Context, Prompt Richness, and Evaluation Controls

Abstract

This paper tests whether full verified-trajectory examples improve software-agent reliability over final-patch examples once evaluation-interface failures are controlled. We use the repaired JSON edit interface from prior work: agents emit structured file edits, the harness validates and repairs the schema, deterministically generates a patch, applies it to a clean repository state, and runs tests. In a corrected 240-run, 30-task pilot, the full verified-trajectory condition solved 55/60 paired condition-runs versus 51/60 for the original final-patch condition. This positive D-over-B signal was small and not statistically established by an exact sign test over discordant paired units ($p = 0.2188$). A token-matched final-patch control then matched the full-trajectory condition at 55/60 solves, showing that prompt richness or verbosity explains much of the original gap. A shuffled-trajectory control solved 52/60, giving ordered trajectories a directional but non-significant edge over shuffled trajectory context ($p = 0.3750$). The resulting conclusion is cautious: verified trajectories remain promising, but this prompt-only pilot does not isolate a trajectory-specific causal effect. Rich, evaluation-aligned context appears sufficient for most of the observed gain.

Core finding
Full verified trajectories beat a short final-patch baseline, but not a token-matched final-patch control. Ordered trajectory structure shows a small directional edge over shuffled trajectory context, but the effect is not statistically established in this pilot.

1. Introduction

Software-repair agents must do more than emit plausible explanations: they must transform a repository into a verified passing state. In prior work in this project, raw diff generation often failed before semantic evaluation was reachable. We therefore introduced a repaired JSON edit interface that separates artifact validity, schema usability, patch application, verified solve, and applied-but-not-verified outcome.

This paper asks the next question: once the interface is stabilized, do verified trajectory examples improve reliability over final successful patch examples? The motivating hypothesis is that agents should benefit from seeing not only final fixes, but also failed attempts, observations, verifier outputs, corrections, and final verified success.

The answer from this pilot is nuanced. Full verified trajectories outperform a short final-patch baseline, but a token-matched final-patch control matches them. This means the pilot supports a rich-context effect more strongly than a clean trajectory-structure effect.

Main contribution

We provide a controlled prompt-level pilot showing how an apparent trajectory benefit can shrink or disappear after prompt-richness controls. The result is useful because it prevents overclaiming: stronger verified-trajectory claims require controls for token count, example density, and instruction specificity.

2. Evaluation Interface and Experimental Unit

All reported Stage 2 and Stage 3 runs use the repaired JSON interface. The model returns a JSON object containing claim_solved and a list of file edits. The harness validates the schema, applies conservative repair where supported, generates a patch deterministically, resets the task repository to a clean committed baseline, applies the patch, and runs tests.

The paired experimental unit for the primary comparisons is task x replication. Each condition is run twice on each of 30 tasks, giving 60 paired units for B-versus-D, B2-versus-D, and D-versus-D-shuffled comparisons. Aggregate rates are reported over condition-runs; paired statistics compare outcomes on matching task and replication indices.

Table 1. Layered evaluation funnel used throughout the study.

Layer	Question answered	Example failure
Raw JSON emitted	Did the model produce a parseable artifact?	invalid_json
Schema usable	Does the output match the expected edit schema?	edits missing or wrong type
Patch generated/applied	Can the edit be converted and applied to the clean repo?	context mismatch or missing path
Verified solve	Do the tests pass after applying the edit?	semantic bug remains
Applied-but-not-verified	Did a patch apply while verification still failed?	applied patch fails tests

3. Prompt Conditions

The study compares a baseline, a final-patch prompt, a full verified-trajectory prompt, and two controls introduced during major revision. The controls are critical: they test whether the observed D-over-B gain is actually due to trajectory structure or simply due to longer, richer context.

Table 2. Prompt conditions and controls.

Condition	Role	Description
Agent A	Instruction-only baseline	Uses the repaired JSON interface with task instructions but no examples or trajectory context.
Agent B	Final-patch baseline	Adds a compact final successful patch pattern, without failed attempts or verifier trajectory.
Agent C	Verified lessons	Adds abstract verification lessons without concrete full trajectories.
Agent D	Full verified trajectory	Adds concrete state-action-observation-verification examples with failures, corrections, and verified success.
Agent B2	Token-matched final patch control	Expands final-patch context to match D-like richness without trajectory failure/correction structure.
Agent D-shuffled	Shuffled trajectory control	Keeps trajectory-like content but disrupts the ordered state -> failure -> observation -> correction sequence.

Token counts are computed from finalized prompt templates using cl100k_base and exclude task-specific issue text and source files.

Prompt	Prompt tokens (cl100k_base)	Examples	Failure/ correction order	Verifier text	Purpose
B	short baseline; not token-matched	final patch pattern	No	minimal	Original compact final-patch baseline.
D	506	verified trajectory	Ordered	Yes	Full trajectory condition.
B2	508	expanded final-patch context	No	limited / non-trajectory	Token-matched rich final-patch control for D.
D-shuffled	630	trajectory-like content	Disrupted	Yes	Trajectory-order control; not exact token-matched to D.

Token counts were computed from finalized prompt templates using cl100k_base and exclude task-specific issue text and source files, which were shared across conditions. The token-matching claim concerns B2 versus D (508 versus 506 tokens). D-shuffled is longer (630 tokens), so it is interpreted as a shuffled trajectory-order control rather than an exact token-matched control. Because D-shuffled is longer than D, its lower solve count is unlikely to be explained by reduced prompt budget, though the comparison still does not isolate ordering perfectly.

4. Stage 2: Corrected 240-Run Expansion

Stage 2 evaluated 30 software-repair tasks, four conditions, and two replications, for 240 condition-task runs. During mechanism analysis, a task-state contamination bug was found: repositories were reset before evaluation but not always cleaned afterward. The contaminated outputs were discarded, the evaluator was fixed to clean task repositories after evaluation, all task repos were reset, and the 240-run expansion was rerun. All Stage 2 results in this draft refer to the corrected frozen run.

Result provenance note
Earlier internal drafts reported lower Stage 2 rates from stale/pre-correction aggregation surfaces. After the evaluator cleanup fix and rerun, the paired verified-solve counts used for the final analysis are 51/60 for Agent B and 55/60 for Agent D. These corrected frozen counts supersede earlier aggregate tables.

This document reports the corrected frozen artifact set only. Earlier lower values should not be mixed with the final paired Stage 2/Stage 3 analysis.

Table 3. Corrected Stage 2 primary comparison. The D-over-B signal is positive but small.

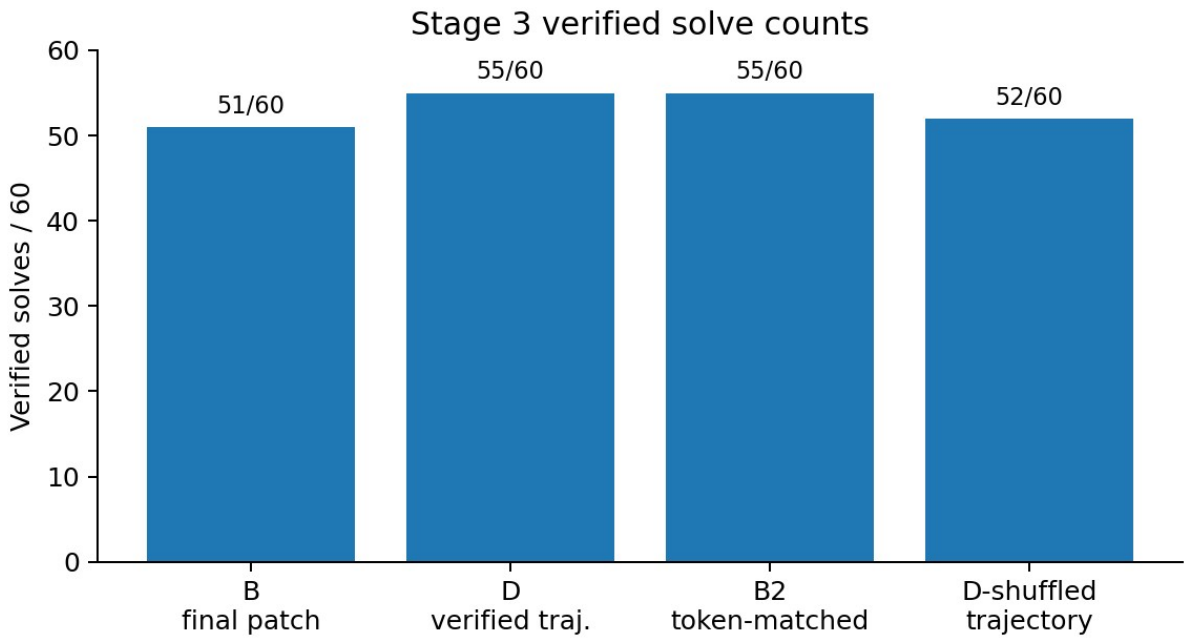
Metric	Agent B	Agent D	D - B	Paired result
Verified solve	51/60	55/60	+4 solves	D-only 5, B-only 1; exact sign p = 0.2188
Task-level comparison	-	-	D wins 4 tasks; B wins 1	24 ties; 1 neither
Mechanism annotation	targeted subset	targeted subset	-	Qualitative probe, not a population recovery estimate

The Stage 2 result supports a modest descriptive advantage for the full verified-trajectory condition over the original final-patch baseline. However, because the effect is only four solves over 60 paired units and the sign test is not significant, the result should be interpreted as a pilot signal rather than a statistically established effect.

5. Stage 3: Prompt-Confound Controls

Stage 3 was added because Agent D differs from Agent B in several ways: it is longer, richer, more explicit, and more verification-oriented. Without controls, a D-over-B difference cannot be attributed specifically to verified trajectory structure. We therefore added two controls: B2, a token-matched final-patch prompt, and D-shuffled, a shuffled-trajectory prompt.

Figure 1. Stage 3 verified solve counts. B2 matches Agent D, while D-shuffled is slightly lower.



For consistency, this revision reports applied-but-not-verified rather than false completion unless explicit claim_solved auditing is part of the row-level analysis. Applied-but-not-verified means that a patch applied but the verifier did not pass.

Table 4. Complete interface and outcome counts for the primary Stage 2 conditions and Stage 3 controls. The last column is applied-but-not-verified, not necessarily audited false completion.

Condition	Verified solves	JSON valid	Schema usable	Patch applied	Applied-but-not-verified
Agent B	51/60	60/60	60/60	53/60	2/60
Agent D	55/60	60/60	60/60	56/60	1/60
Agent B2	55/60	60/60	60/60	60/60	5/60
Agent D-shuffled	52/60	59/60	59/60	59/60	7/60

Figure 2. Paired solve-count differences. All observed effects are small in this pilot.

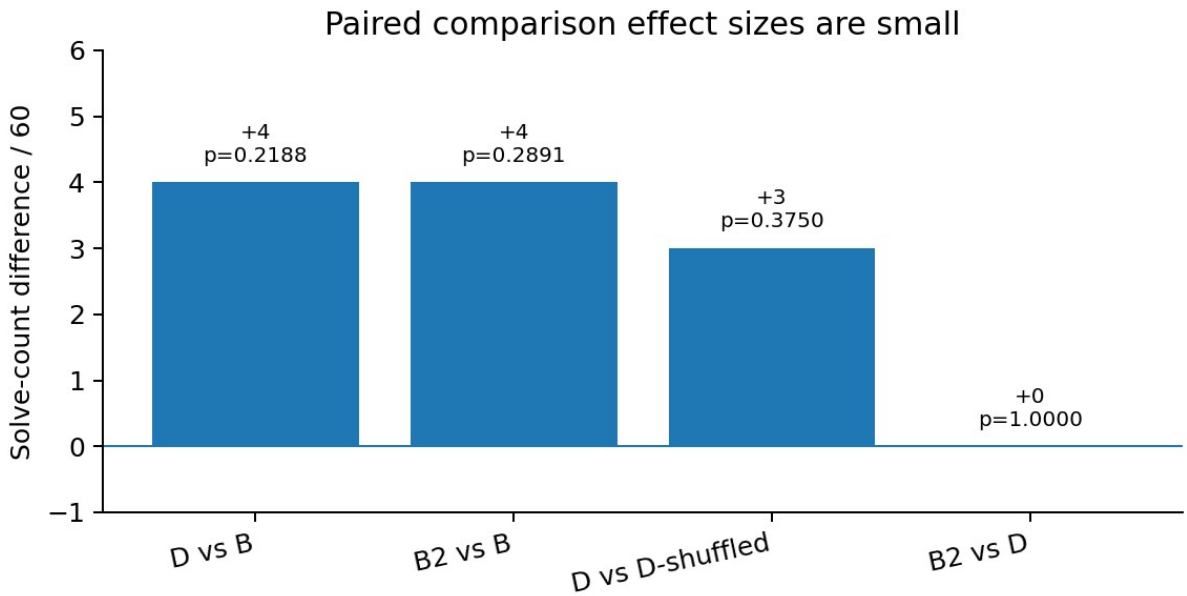


Table 5. Stage 3 paired comparisons. Left-only and right-only are discordant paired task-replication units; ties include both-true and both-false pairs.

Comparison	Left solves	Right solves	Left-only	Right-only	Ties	Exact sign p
D vs B	55/60	51/60	5	1	54	0.2188
B2 vs B	55/60	51/60	6	2	52	0.2891
B2 vs D	55/60	55/60	1	1	58	1.0000
D vs D-shuffled	55/60	52/60	4	1	55	0.3750

Comparison	Verified-solve difference	95% paired bootstrap CI	Exact sign p	Interpretation
D vs B	+4/60 (+0.067)	[0.000, 0.150]	0.2188	Positive pilot signal, not established.
B2 vs B	+4/60 (+0.067)	[-0.017, 0.167]	0.2891	Richer final-patch context improves baseline directionally.
B2 vs D	0/60 (0.000)	[-0.050, 0.050]	1.0000	B2 matches D on solves.
D vs D-shuffled	+3/60 (+0.050)	[-0.017, 0.133]	0.3750	Ordered trajectories directionally exceed shuffled content.

Exact p-values use a two-sided exact binomial sign test over discordant task x replication pairs only. Paired bootstrap intervals below resample the 60 paired units with replacement. They describe uncertainty over the observed pilot units, not a general task-distribution confidence interval.

Interpretation: D has a positive pilot signal over the short B baseline; B2 eliminates the observed D advantage; and ordered trajectories have only a small directional edge over shuffled trajectory content.

Stage 3 interpretation

The B2 result is the key ablation. Because B2 matches D, the pilot does not isolate a trajectory-specific effect. The D-shuffled result leaves open a possible small benefit from ordered trajectory structure, but the evidence remains directional rather than statistically established.

B2 achieved perfect patch applicability (60/60) but did not exceed Agent D on verified solves. This suggests that token-matched final-patch context improved interface reachability, while semantic correctness after reaching the verifier remained the limiting factor in the remaining failures.

6. Recovery Annotation and Mechanism Analysis

The motivating mechanism for verified trajectories is recovery: the agent should use failure or verifier evidence to correct an initially wrong path. However, the retained artifacts mostly store final structured outputs rather than complete reasoning traces. Recovery annotation was therefore applied only to a prioritized subset of differential and false-completion cases.

This annotation should be interpreted as a targeted qualitative probe, not as a random sample and not as an estimate of population recovery frequency. In the reviewed subset, only one D-over-B case was coded as recovery-like. Most differential cases looked more like better direct patch generation or generation reliability than observable stepwise recovery.

Table 6. Mechanism analysis after corrected Stage 2 and Stage 3 controls.

Finding	Interpretation
One recovery-like D-over-B case identified	Trajectory examples may sometimes help agents avoid shallow fixes, but evidence is sparse.
Several D wins look like direct generation reliability	The main advantage may be producing a usable edit, not recovering through visible verifier feedback.
challenge_003 no longer differential after corrected rerun	The earlier anecdote should not be used as strong mechanism evidence.
22/240 rows annotated in Stage 2D subset	Targeted qualitative probe only; not a complete recovery-rate estimate.

7. Discussion

7.1 What the pilot supports

The pilot supports three claims. First, full verified trajectories outperform a short final-patch baseline descriptively. Second, token-matched final-patch context can match the full verified-trajectory condition, making prompt richness a major confound. Third, ordered trajectory examples may have a small directional advantage over shuffled trajectory examples, but this pilot does not establish that effect statistically.

The practical lesson is that prompt-level studies of trajectories must include controls for length, example density, operational specificity, and interface alignment. Otherwise, a trajectory advantage may simply be a rich-context advantage.

7.2 What the pilot does not prove

This study does not prove verified experience scaling. It does not fine-tune models, vary trajectory data volume, measure scaling laws, or establish a causal trajectory-structure effect. It tests prompt-level conditioning in one software-repair environment and one model/backend path. The result is valuable because it narrows the hypothesis and identifies the confounds that a stronger study must control.

7.3 Revised answer to the title question

Do verified trajectories improve software-agent reliability? In this pilot: maybe, but not cleanly yet. Verified trajectories beat a short final-patch baseline, but not a token-matched final-patch control. Ordered trajectory structure has a small directional edge over shuffled trajectory content, but the evidence is not conclusive.

8. Limitations

Table 7. Main limitations.

Limitation	Why it matters
Prompt-only study	No training-time scaling or parameter updates are tested.
Single model/backend path	Findings may depend on the model and ollama-cli transport.
Synthetic 30-task benchmark	Tasks are useful for controlled pilots but not a broad external benchmark.
Prompt confounds remain	B2 controls length/richness partially, but not every dimension of examples and instruction wording.
Partial mechanism annotation	Recovery analysis is a qualitative probe over selected rows, not a full frequency estimate.
Small effect sizes	D-over-B and D-over-shuffled differences are descriptive and not statistically established.

9. Future Work

The next study should move from prompt-level controls to stronger causal tests. The most important experiment is a matched factorial design varying four factors independently: token budget, final-patch examples, ordered trajectories, and verifier/correction ordering. A stronger version would fine-tune or train on verified trajectory data at multiple scales and evaluate on held-out repositories.

Table 8. Future work needed to isolate trajectory-specific effects.

Next experiment	Purpose
D-no-guidance	Remove extra operational advice from trajectory examples to isolate example structure.
B2 plus shuffled non-trajectory context	Control for length and generic rich context.
Ordered vs shuffled trajectories across models	Test whether trajectory order matters beyond one backend.
Training-time VET scaling	Move from prompt-level evidence to true scaling evidence.
External benchmark tasks	Test whether findings transfer beyond synthetic challenge tasks.

10. Conclusion

The original hypothesis was that verified trajectories improve software-agent reliability by exposing agents to state-action-observation-verification structure. The pilot result is more cautious. Full verified trajectories did outperform a short final-patch baseline, but a token-matched final-patch control matched them, and a shuffled-trajectory control trailed only slightly. The best conclusion is that rich, evaluation-aligned prompt context improves reliability, while the specific contribution of ordered verified trajectory structure remains unresolved.

This is still a useful result. It identifies the controls required before claiming verified-trajectory benefits and shows why prompt-richness confounds matter in agent-evaluation studies. A future verified-experience scaling paper should build on this lesson rather than skip it.

Appendix A. Stage Timeline

Table A1. Project timeline.

Stage	Status	Role
Stage 0	Passed	12-run tiny check across three tasks and four conditions.
Stage 1	Passed / frozen	80-run pilot over 10 tasks.
Stage 2A/B	Passed	20 new tasks authored and gold/empty controls validated.
Stage 2C	Passed / frozen after correction	240-run, 30-task expansion rerun after evaluator cleanup fix.
Stage 2D	Passed / partial	Targeted mechanism annotation over selected rows.
Stage 3A/B/C	Passed	Paired statistics, task-level table, and recovery wording correction.
Stage 3D	Passed	B2 token-matched final-patch control.
Stage 3E	Passed	D-shuffled trajectory control.

Appendix B. Prompt-Control Summary

The prompt files are part of the artifact package. Agent B2 intentionally controls for prompt richness by expanding final-patch context without including failed-attempt or verifier-correction trajectory structure. Agent D-shuffled preserves trajectory-like ingredients but disrupts the chronological state-action-verification order. These controls change the interpretation of the D-over-B signal from trajectory-specific to context-sensitive.

Appendix C. Canonical Artifacts

Table C1. Canonical artifacts for this revision.

Artifact	Purpose
results/vet_scaling_stage2/stage2_freeze.md	Corrected Stage 2 freeze note.
results/vet_scaling_stage2/summary_by_condition.csv	Corrected Stage 2 aggregate condition metrics.
results/vet_scaling_stage3/paired_b_vs_d_stats.csv	Paired D-vs-B statistics.
results/vet_scaling_stage3/task_level_b_vs_d_full.csv	Full 30-task B-vs-D table.
results/vet_scaling_stage3/b2_summary.csv	B2 token-matched control summary.
results/vet_scaling_stage3/b2_vs_d_stats.csv	B2 versus D paired comparison.

results/vet_scaling_stage3/b2_vs_b_stats.csv	B2 versus B paired comparison.
results/vet_scaling_stage3/d_shuffled_summary.csv	D-shuffled control summary.
results/vet_scaling_stage3/d_vs_d_shuffled_stats.csv	D versus D-shuffled paired comparison.
results/vet_scaling_stage3/stage3_summary.md	Stage 3 synthesis and final interpretation.

References

1. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv, 2023.
2. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv, 2022.
3. Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv, 2023.
4. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv, 2023.
5. Chen et al. Teaching Large Language Models to Self-Debug. arXiv, 2023.
6. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.
7. Lightman et al. Let's Verify Step by Step. arXiv, 2023.
8. Deng et al. Mind2Web: Towards a Generalist Agent for the Web. NeurIPS, 2023.
9. Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv, 2023.
10. Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv, 2024.